Final Report

Advanced Software Product Development:

Internship Assignment Optimization System

Team 3

Hedi Sayadi
Anis Belkacem
Hadir Lakhal
Fekher Salem
Youssef Moustafa

Supervisor: Benedikt Fein

WiSe 2025/26

February 2026

Management Summary

Client and Context

This project was developed for the University of Passau, specifically for the *Zentrum für Lehrkräftebildung und Fachdidaktik* (Center for Teacher Education and Subject Didactics), *Praktikumsamt für Grund- und Mittelschule* (Internship Office for Elementary and Middle Schools).

Each year, this unit is responsible for planning and organizing teaching internships for university students in collaboration with elementary and middle schools throughout the region. This planning process is time-consuming and complex, requiring careful coordination between student requirements, school capacities, and available supervising teachers. The office sought an automated solution to streamline this process while maintaining the quality and feasibility of internship assignments.

Core Challenges

The internship planning process faces several significant challenges:

- **Large number of students:** Hundreds of students require placements each semester, making manual planning impractical
- **Different internship types and subjects:** Multiple types of internships with varying subject requirements must be coordinated
- **Complex constraints:** Numerous rules govern valid assignments, including teacher qualifications, workload limits, and school type matching
- **Distance and accessibility:** Geographic considerations affect both student convenience and teacher availability
- **Overall budget constraints:** The total number of active internship placements is limited by available resources

Solution Delivered

A comprehensive web-based software system that:

- **Automates assignment optimization:** Systematically assigns teachers and students to internship placements while respecting all constraints

- **Multi-phase planning process:** Separates teacher assignment from student assignment, reflecting real-world planning workflow
- **Configurable parameters:** Allows adjustment of budget limits, constraint priorities, and optimization goals
- **Interactive management interface:** Provides tools for data input, assignment review, and manual adjustments when needed

Outcomes

The system delivers tangible benefits to the internship planning process:

- **High-quality, feasible assignment plans:** Produces assignments that satisfy hard constraints while optimizing for quality preferences
- **Customizable solutions:** Maintains flexibility for manual adjustments and what-if scenario planning
- **Reduced manual effort:** Significantly decreases planning time once initial data is entered

Contents

Chapter 1

Introduction

1.1 Teaching Internships in Teacher Education

Practical teaching experience is a cornerstone of teacher education programs. While theoretical coursework provides foundational knowledge in pedagogy, child development, and subject-specific didactics, it is through hands-on classroom experience that future teachers develop essential professional competencies. Teaching internships bridge the gap between academic study and professional practice, offering students the opportunity to observe experienced educators, experiment with teaching methods, and receive mentored feedback in authentic school settings.

At the University of Passau, education students pursuing elementary and middle school teaching credentials are required to complete four distinct internship types throughout their program of study:

- **PDP I** (*Pädagogisch-didaktisches Praktikum I*): An initial pedagogical-didactic internship introducing students to classroom observation and basic teaching interactions
- **ZSP** (*Zusätzlich studienbegleitendes Praktikum*): An additional internship accompanying coursework, focusing on subject-specific teaching methods
- **PDP II** (*Pädagogisch-didaktisches Praktikum II*): An advanced pedagogical-didactic internship with increased teaching responsibilities
- **SFP** (*Studienbegleitendes fachdidaktisches Praktikum im Unterrichtsfach*): A subject-didactic internship in a specific teaching subject, integrating theory and practice

Each internship type serves distinct pedagogical goals and occurs at different stages of the degree program. Successful completion of all four internships is mandatory for graduation and licensure as a teacher. Critically, these internships take place in real elementary and middle schools throughout the region, supervised by active classroom teachers who mentor students and evaluate their performance.

1.2 Scale and Complexity of Internship Coordination

The logistical challenge of organizing these internships is substantial. For the 2025/2026 academic year, the *Praktikumsamt* must coordinate approximately **420 planned internship placements** across roughly **100 schools** distributed among **8 regional school districts** (*Schulämter*). Each school employs multiple teachers with varying subject specializations, availability, and capacity to supervise student interns.

The assignment process must account for numerous factors:

- **Student requirements:** Matching each student's internship type, subject specialization, and school type
- **Teacher qualifications and preferences:** Supervising teachers have individual preferences regarding which internship types and subject areas they wish to supervise
- **School characteristics:** Distinguishing between elementary schools (*Grundschule*, GS) and middle schools (*Mittelschule*, MS)
- **Student geographic accessibility:** For PDP I and PDP II internships, students must be assigned to schools close to their home addresses to minimize commute time
- **Capacity constraints:** Respecting internship capacity limits and institutional budget restrictions on the total number of active placements

1.3 Limitations of Manual Planning

Historically, internship assignments have been managed through manual processes involving spreadsheets, email coordination, and institutional knowledge. While this approach has functioned adequately in the past, it presents significant limitations:

- **Time intensity:** Manually matching hundreds of students to appropriate placements requires weeks of dedicated effort by administrative staff
- **Suboptimal assignments:** Without systematic optimization, manual assignments may inadvertently result in long commutes for students, unbalanced teacher workloads, or missed opportunities for better matches
- **Error susceptibility:** Human planners may overlook constraint violations (e.g., assigning a teacher beyond their capacity, mismatching subject requirements) or introduce data entry errors
- **Lack of transparency:** It is difficult to explain why particular assignments were made or to demonstrate that assignments are fair and equitable
- **Inflexibility:** When circumstances change (e.g., a teacher becomes unavailable, enrollment projections shift), revising assignments manually is labor-intensive and may cascade into broader disruptions

- **Limited optimization:** Manual processes struggle to balance competing objectives such as minimizing student travel distance while preserving teacher assignment continuity across semesters

These limitations motivated the *Praktikumsamt* to seek an automated solution that could systematically generate high-quality assignments while reducing administrative burden.

1.4 Course Context

This project was developed as part of the **Advanced Software Product Development (ASPD)** course at the University of Passau during the winter semester of 2025/2026. The ASPD course is structured around real-world client projects, providing students with practical experience in collaborative software engineering.

The course operates with the following organizational structure:

- **Four student teams:** Each team consists of five students, for a total of 20 participants
- **Shared client need:** All four teams address the same problem domain (internship planning), but may approach implementation differently or use different technologies
- **Agile methodology:** Development follows an iterative, sprint-based approach with **five sprints**, each lasting **two weeks**, for a total development period of ten weeks
- **Teaching assistant supervision:** Each team is guided by a dedicated teaching assistant (TA) who provides mentorship, oversight, and coordination with the client
- **Client engagement:** Communication with the *Praktikumsamt* ensures that requirements are grounded in real needs and that delivered solutions are practically deployable

This structure creates a realistic software development environment where students must balance technical challenges, evolving requirements, team coordination, and stakeholder expectations—mirroring professional software engineering practice.

1.5 Team Contribution

Our team developed a complete, standalone internship assignment system. While all four teams in the course addressed the same client need, each team worked independently to create their own implementation from scratch. Our specific approach and contributions included:

- **Constraint modeling:** Translating the complex business rules governing valid assignments into formal hard and soft constraints

- **Multi-phase optimization architecture:** Designing a phased approach that separates teacher assignment from student assignment to manage computational complexity
- **OptaPlanner integration:** Implementing constraint satisfaction solving using the OptaPlanner framework
- **Score calculation logic:** Developing efficient algorithms to evaluate solution quality based on distance minimization, workload balance, and assignment diversity
- **Backend API development:** Creating RESTful endpoints for data management, solver invocation, and result retrieval
- **Frontend interface:** Building a complete web-based user interface for system interaction

Chapter 2

Problem Definition & Requirements

2.1 System Vision

The internship assignment system is designed to support the *Praktikumsamt's* annual planning cycle while maintaining flexibility and quality. The core vision rests on three pillars:

- **Systematic assignment optimization:** Rather than relying on manual processes, the system applies algorithmic techniques to systematically generate assignments that satisfy constraints and optimize for quality
- **Real-world pragmatism:** Assignments must respect practical constraints including teacher availability, student location preferences, and institutional budgets. The system prioritizes *solution quality* over exhaustive feasibility guarantees, acknowledging that in complex problems with many competing objectives, perfect feasibility may be unattainable
- **Operational transparency:** The system maintains an audit trail of assignments and provides clear justification for decisions, enabling the *Praktikumsamt* to explain assignments to stakeholders and make adjustments when needed

2.2 Stakeholders

2.2.1 Primary: Internship Coordination Unit (*Praktikumsamt*)

The internship office is the primary stakeholder and system user. They require:

- Efficient tools to coordinate hundreds of placements annually, reducing time spent on manual assignment work
- Flexibility to review and manually adjust assignments when business circumstances change

2.2.2 Secondary: Teaching Students

Students require high-quality placements that serve their educational development. Their interests include:

- Geographic accessibility: for PDP I and PDP II internships, assignment to schools near their home addresses to make regular school visits feasible
- Subject-appropriate placements aligned with their intended teaching specialization

2.2.3 Secondary: Schools and Supervising Teachers

Schools and their supervising teachers are the infrastructure that enables internships. Their interests include:

- Balanced workload: not overwhelming individual teachers with too many interns while ensuring adequate supervision
- Alignment with subject expertise and teaching preferences (teachers have preferences for which internship types and subjects to supervise)
- Respect for institutional capacity constraints and available supervision time

2.3 Core Problem Definition

The internship assignment problem can be formally defined by specifying inputs, constraints, and objectives.

2.3.1 Input Data

The system operates on four categories of input:

1. **Students:** Each student record contains:
   - Internship type requirement (PDP I, PDP II, ZSP, or SFP for a given semester)
   - Subject specialization (primary teaching subject)
   - Home address (for distance calculations)
2. **Teachers:** Each teacher record contains:
   - Subject expertise and qualifications
   - Preferences: which internship types and subjects they are willing to supervise
   - Maximum internship capacity (workload limit per semester/year)
   - Availability constraints
3. **Schools:** Each school record contains:
   - Type: elementary (*Grundschule*, GS) or middle school (*Mittelschule*, MS)
   - Geographic location (address and coordinates)
   - Zone and public transport accessibility

- List of supervising teachers and their availability

4. **Institutional constraints:** System-level parameters including:
   - Total budget: maximum number of active internship placements for the semester/year

2.3.2 Constraints

The system must respect multiple categories of constraints representing real-world business rules and stakeholder requirements:

Hard Constraints (must be satisfied)

Hard constraints are non-negotiable requirements reflecting institutional policies and operational feasibility:

- **Budget constraint:** The total number of scheduled internship placements cannot exceed the institutional budget limit for the academic year
- **Complete student assignment:** Every student must receive exactly one internship placement matching their internship requirements
- **Teacher workload limits:** Each teacher can supervise 0, 1, 2, or 4 internships per planning cycle (semester or year, depending on internship type).[1]
- **Teacher max workload:** Each teacher can specify a maximum number of internships they prefer not to exceed
- **Internship capacities:** PDP internships can accommodate at maximum two students, SFP and ZSP can accommodate at maximum 4
- **Teacher diversity requirement:** If a teacher supervises multiple internships, they must be of different types. A teacher cannot supervise multiple instances of the same internship type (e.g., two PDP I internships)
- **Subject qualification:** Teachers supervising ZSP or SFP internships (which are subject-specific) must have relevant subject expertise. SFP internships must match the student's main course
- **School type matching:** Students pursuing elementary school certification must be assigned to elementary schools (GS), and those pursuing middle school certification to middle schools (MS)
- **School zones matching:** Internships, by type, must take place in schools with specific zones
- **Teacher availability:** Internship placements can only be scheduled at schools with available, qualified supervising teachers

[1] The *Praktikumsamt_Uebersicht* documentation specifies that teachers should be assigned exactly 2 internships. However, analysis of historical data (example_data) reveals teachers with 1, 2, and 4 internship assignments, with 2 being the most common configuration. The system accommodates this flexibility while treating 2 internships as the preferred workload through soft constraint optimization.

Soft Constraints (preferences to optimize)

Soft constraints are desirable properties that improve assignment quality and stakeholder satisfaction:

- **Student proximity for PDP I & PDP II:** Minimize the geographic distance between assigned students and their home addresses. This ensures practical feasibility for repeated school visits
- **Teacher workload preference:** Prefer assigning teachers exactly 2 internships over other valid counts (1, 4, or 0). This represents ideal teacher engagement
- **Diversity at schools:** Prefer schools with diverse internship types, rather than schools where one type dominates
- **Teacher preference matching:** Favor assignments of internship types and subjects that align with individual teacher preferences
- **Teacher assignment preservation:** When reoptimizing across semesters, prefer preserving teacher assignments for the same types of internships. This promotes continuity and relationship-building between teachers and the university program
- **Subject Matching for ZSP:** Unlike SFP, ZSP internships are flexible regarding the subject course, favor assignments that align with students preferences (main course + 3 additional preferred courses)

2.3.3 Objectives

The system has two primary objectives:

1. **Feasibility:** Generate assignment plans that satisfy all hard constraints. This ensures that assignments are valid and implementable
2. **Optimization:** Among feasible solutions, select plans that maximize soft constraint satisfaction. This means minimizing student travel distances, balancing teacher workloads, respecting preferences, and achieving other quality goals

Importantly, **this system is an optimization system rather than a pure feasibility solver**. In problems with many competing constraints and objectives, guaranteed feasibility across all scenarios may be unattainable. The system prioritizes generating *high-quality, near-feasible* solutions over mathematical certainty of feasibility. This pragmatic approach reflects real-world practice: even if 100% feasibility cannot be assured, the system can produce solutions that satisfy most constraints and provide a high-quality starting point for manual adjustment if needed.

Feasibility Landscape

A key observation mitigates feasibility concerns: the institution has a favorable constraint-to-resource ratio. The total number of available supervising teachers substantially exceeds the number of students requiring placements. For the 2025/2026 academic year, approximately 100 schools with multiple teachers each, must accommodate roughly

420 planned internships. This surplus of teacher availability significantly improves the likelihood of finding feasible solutions in most scenarios.

However, the system is deliberately structured to strategically satisfy constraints even when full satisfaction is impossible, rather than assuming feasibility is guaranteed. This defensive design approach:

- Acknowledges that specific combinations of constraints (e.g., specialized subject requirements + teacher preferences) can create local infeasibility despite overall abundance
- Implements constraint prioritization so that violations, if they must occur, affect lower-priority (preference) constraints rather than critical hard (impossibility) constraints
- Ensures robust behavior even in edge cases or future scenarios with tighter resource availability

2.4 System Characteristics

2.4.1 Deterministic Optimization

The system uses constraint satisfaction and local search optimization to explore the solution space. While it does not guarantee globally optimal solutions (such problems are NP-hard), it consistently produces high-quality solutions within acceptable runtime limits.

2.4.2 Configurability

The *Praktikumsamt* can adjust:

- Budget limits per semester
- Teacher preferences and availability

2.4.3 Interactive Assignment Management

While the core optimization engine operates automatically, the system provides interfaces for:

- Reviewing and validating proposed assignments
- Making manual adjustments when needed (e.g., if a teacher becomes unavailable)

Chapter 3

System Architecture

3.1 Architectural Overview

3.1.1 Three-Tier Architecture

The system follows a classic three-tier architecture separating presentation, business logic, and data persistence:

- **Presentation Tier (Frontend):** Web-based user interface built with React and TypeScript, providing interactive management of students, teachers, schools, and assignments
- **Business Logic Tier (Backend):** Spring Boot application containing optimization engine, constraint validation, and business rules
- **Data Tier:** PostgreSQL relational database for persistent storage of all entities and assignment results

3.1.2 Design Principles

The architecture adheres to several key principles:

- **Separation of concerns:** Clear boundaries between data access, business logic, optimization, and presentation
- **RESTful API:** Stateless HTTP API enables frontend integration and potential future extensions (mobile apps, third-party integrations)
- **Backend-centered optimization:** Computationally intensive constraint solving runs server-side to leverage more powerful hardware
- **Asynchronous processing:** Long-running optimizations execute asynchronously with job status polling to avoid blocking the user interface

3.2 Technology Stack

3.2.1 Backend Technologies

Core Framework:

- **Spring Boot 3.x:** Provides dependency injection, REST controllers, transaction management, and production-ready features
- **Spring Data JPA:** Simplifies database access through repository pattern and object-relational mapping
- **Spring Security:** Handles authentication and authorization for role-based access control

Optimization Engine:

- **OptaPlanner 9.x:** Constraint satisfaction solver implementing local search metaheuristics (Tabu Search, Late Acceptance, Simulated Annealing)
- Embedded within Spring Boot application for tight integration

Database:

- **PostgreSQL 15:** Relational database chosen for reliability, ACID compliance, and advanced query capabilities
- Supports complex queries for reporting and analytics

Build System:

- **Maven:** Dependency management and build automation

3.2.2 Frontend Technologies

Core Framework:

- **React 18:** Component-based UI library for building interactive interfaces
- **TypeScript:** Type-safe JavaScript providing compile-time error detection and improved IDE support

Build Tools:

- **Vite:** Fast development server and optimized production builds
- Hot module replacement for rapid development iteration

UI Components:

- Custom React components for tables, forms, modals, and visualizations
- CSS modules for scoped styling

3.2.3 Development Tools

- **Git/GitLab:** Version control and code review
- **Docker:** Containerization for consistent development and deployment environments
- **Postman:** API testing and documentation

3.3 System Layers and Components

3.3.1 Data Access Layer

Entities:

- `Student`, `Teacher`, `School`, `Course`: Core master data entities
- `StudentConfig`: Configures which internship types a student needs for a specific semester
- `PlannedInternship`: Represents an internship slot (planning entity for Phase 1)
- `StudentInternshipDemand`: Represents a student's need for a specific internship (planning entity for Phase 2)
- `InternshipAssignment`: Final assignment linking student, teacher, school, and internship type
- `BaselineAssignment`: Captured assignments for semester continuity preservation

Repositories:

- Spring Data JPA repositories for each entity
- Custom query methods for complex lookups (e.g., finding teachers by subject expertise and availability)

3.3.2 Service Layer

Master Data Services:

- `StudentService`, `TeacherService`, `SchoolService`: CRUD operations for master data
- Data validation and business rule enforcement

Optimization Services:

- `Phase1OptimizationService`: Teacher assignment and slot activation
- `Phase2OptimizationService`: Student assignment to activated slots
- `ReoptimizationService`: Semester-to-semester continuity with baseline preservation

Supporting Services:

- `BaselineService`: Captures and retrieves baseline assignments for preservation
- `OptimizationJobService`: Manages asynchronous job status and progress tracking
- `AuditLogService`: Records system actions for transparency and debugging

3.3.3 Controller Layer

RESTful controllers expose endpoints for:

- Master data management (students, teachers, schools, courses)
- Solver invocation (Phase 1, Phase 2, reoptimization)
- Job status polling for asynchronous operations
- Assignment retrieval and export
- System configuration

Example endpoints:

- `POST /api/internships/phase1/optimize-async`: Start Phase 1 optimization
- `GET /api/jobs/{jobId}`: Poll optimization job status
- `GET /api/assignments?schoolYear=WiSe25-26`: Retrieve assignments for a semester

3.3.4 Frontend Layer

Page Components:

- Dashboard: Overview of assignments and system status
- Students, Teachers, Schools: Master data management interfaces
- Assign: Optimization control panel with Phase 1/2/reoptimization triggers
- Audit Logs: System activity history

Shared Components:

- Tables with sorting, filtering, and pagination
- Forms with validation and error handling
- Modals for data entry and confirmation dialogs
- Status indicators for optimization progress

3.4 Core Data Model

3.4.1 Entity Relationships

The data model captures the following key relationships:

- **Student ← StudentConfig:** One-to-many (a student has configurations for multiple semesters)
- **Teacher ← School:** Many-to-one (a teacher works at one school)

- **PlannedInternship → Teacher:** Many-to-one (multiple internships can be supervised by one teacher)
- **PlannedInternship → Course:** Many-to-one (for subject-specific internships)
- **StudentInternshipDemand → StudentConfig:** Many-to-one (multiple demands per student config, one per internship type)
- **StudentInternshipDemand → PlannedInternship:** Many-to-one (assignment relationship determined by solver)
- **InternshipAssignment:** Links student, teacher, school, and internship type in final assignment

3.4.2 Key Attributes

Student:

- Matriculation number (unique identifier)
- Name and contact information
- Home address (geocoded for distance calculations)
- Semester address (if different from home)

Teacher:

- Subject expertise (list of courses they can supervise)
- Preferences (which internship types and subjects they prefer)
- Maximum workload (internship capacity limit)
- Active status (availability toggle)

School:

- Type (Grundschule or Mittelschule)
- Geographic coordinates (latitude/longitude)
- Zone and public transport accessibility (OEPNV classification)
- Active status

PlannedInternship (Phase 1 planning entity):

- Praktikum type (PDP_I, PDP_II, ZSP, SFP)
- School type (GS or MS)
- Active flag (whether this slot is opened)
- Assigned teacher (planning variable)
- Course (planning variable for ZSP, fixed for SFP, null for PDP)

- Capacity (2 for PDP, 4 for ZSP/SFP)

StudentInternshipDemand (Phase 2 planning entity):

- Student configuration reference
- Praktikum type requirement
- Subject preferences (for ZSP course matching)
- Assigned internship (planning variable)
- Baseline internship (for preservation during reoptimization)
- Pinned flag (hard constraint to preserve specific assignments)

3.5 Multi-Phase Assignment Strategy

The assignment process is deliberately structured into multiple phases to manage computational complexity and reflect real-world planning workflows:

3.5.1 Phase 0: Data Preparation

Before optimization begins, the system prepares and validates input data:

- **Demand slot generation:** For each student configuration, create one `PlannedInternship` slot per required internship type
- **Address geocoding:** Convert addresses to coordinates for distance calculations
- **Validation:** Check for missing data, invalid references, and constraint violations
- **Aggregation:** Calculate demand centroids by internship type and school type for geographic optimization

Rationale: Separating preparation from optimization ensures clean, validated input and enables reuse of demand data across multiple optimization runs.

3.5.2 Phase 1: Teacher Assignment and Slot Activation

Goal: Decide which internship slots to activate (within budget) and assign supervising teachers to each active slot.

Planning variables (what OptaPlanner decides):

- For each `PlannedInternship`: `active` (boolean), `assignedTeacher` (Teacher or null), `course` (for ZSP)

Rationale: This phase handles capacity planning (how many internships to offer) and teacher workload balancing before individual student assignment. It reduces complexity by not yet considering individual student preferences.

3.5.3 Phase 2: Student Assignment

Goal: Assign each student to an activated internship slot, minimizing distance and respecting subject preferences.

Planning variables:

- For each `StudentInternshipDemand`: `assignedInternship` (PlannedInternship or null)

Rationale: With teacher availability and slot activation already determined, Phase 2 focuses purely on student satisfaction. This separation dramatically reduces search space complexity.

3.5.4 Reoptimization with Baseline Preservation

For semester-to-semester transitions (e.g., WiSe25-26 → SoSe26), the system supports reoptimization that preserves teacher and student assignments when possible:

- **Baseline capture:** After completing Phase 1 and Phase 2 for a semester, capture current assignments as baseline
- **Baseline application:** When reoptimizing for the next semester, load previous baseline and apply soft constraints rewarding preservation
- **Pinning mechanism:** Optionally pin specific assignments as hard constraints (e.g., if a teacher-student relationship should be preserved unconditionally)

Rationale: Continuity benefits both teachers (familiarity with university expectations) and students (relationship building). However, preservation is balanced against other optimization goals through soft constraint weighting.

Chapter 4

Implementation Details

4.1 Phase 1 Implementation: Teacher Assignment

4.1.1 Optimization Goal

Assign supervising teachers to internship slots, activate an optimal subset of slots within budget, and maximize coverage while respecting capacity constraints.

4.1.2 Planning Variables

For each PlannedInternship slot:

- `assignedTeacher` (Teacher or null)
- `active` (boolean)
- `course` (for flexible course assignments like ZSP)

4.1.3 Constraint Implementation

4.1.4 Constraint Implementation

Hard constraints:

- Budget: total active slots $\leq$ budget limit
- Teacher workload: 0, 1, 2, or 4 internships per teacher
- Teacher diversity: if 2+ internships, all must be different types
- Subject qualification: teachers must be qualified for assigned courses
- Zone feasibility: appropriate zones per internship type
- Course pinning: PDP has no course, SFP course is fixed

Soft constraints:

- Prefer 2 internships per teacher
- Minimize PDP distance (teachers to student demand centroids)

- Diversity: varied internship types at each school
- Preserve teacher assignments from previous semester

4.1.5 Distance Optimization for PDP

Two mechanisms ensure geographic accessibility:

1. **Bidirectional matching:** Each student finds the closest teacher, and each teacher finds the closest student
2. **Centroid distance:** Teachers are positioned toward student demand centers

This ensures geographic spread and practical accessibility for PDP students.

4.1.6 Trade-offs

Balance coverage vs. budget, trade off teacher preference vs. student demand, optimize for quality (may not achieve strict feasibility in all scenarios).

4.2 Phase 2 Implementation: Student Assignment

4.2.1 Optimization Goal

Assign individual students to activated slots, minimize total distance for PDP students, and respect subject preferences.

4.2.2 Planning Variables

For each StudentInternshipDemand:

- `assignedInternship` (PlannedInternship or null)

4.2.3 Constraint Implementation

Hard constraints:

- Each student assigned to exactly one slot
- Slot must match student's internship type requirement
- Slot must be active (from Phase 1)
- Subject matching for ZSP/SFP

Soft constraints:

- Minimize distance for PDP students to assigned school
- Prefer slots matching student subject preferences

4.2.4 Rationale

Separation from Phase 1 reduces complexity, teacher availability is already determined, focus is purely on student satisfaction, and enables what-if scenarios without reoptimizing teachers.

4.3 Score Calculator Implementation

4.3.1 Design Decision: EasyScoreCalculator

Due to OptaPlanner caching complexities, we implemented:

- Manual `EasyScoreCalculator` to avoid caching issues
- Pure Java constraint evaluation for transparency
- Efficient scoring logic to handle large problem instances
- Detailed logging for debugging and validation

4.3.2 Score Calculation Process

For each solution, the score calculator:

1. Iterates through all planning entities
2. Evaluates each constraint
3. Accumulates hard and soft score penalties
4. Returns a HardSoftScore object

4.4 Solver Configuration

4.4.1 Configuration Parameters

Key configuration parameters:

- Local search algorithms (Tabu Search for Phase 1, Late Acceptance for Phase 2)
- Time budget per optimization run (configurable via UI)
- Score weight tuning to balance hard vs. soft constraints

4.4.2 Algorithm Selection

Phase 1: Tabu Search

- Good for avoiding cycling in complex constraint spaces
- Effective for capacity planning problems

Phase 2: Late Acceptance

- Efficient for assignment problems with many entities
- Good balance between exploration and exploitation

4.5 Asynchronous Processing

4.5.1 Job Management

Long-running optimizations execute asynchronously:

- Backend spawns optimization in separate thread
- Returns job ID immediately to frontend
- Frontend polls job status endpoint periodically
- Progress updates displayed to user

4.5.2 Status Tracking

Job statuses:

- `QUEUED`: Waiting to start
- `RUNNING`: Optimization in progress
- `COMPLETED`: Successfully finished
- `FAILED`: Error occurred

4.6 Data Validation and Error Handling

4.6.1 Input Validation

Before optimization:

- Check for missing required data (students without addresses, teachers without qualifications)
- Validate budget constraints
- Verify school and teacher availability
- Detect potential infeasibility (e.g., insufficient qualified teachers for specific subjects)

4.6.2 Error Recovery

When errors occur:

- Clear error messages displayed to user
- Detailed logs for debugging
- Partial results saved when possible
- System state rolled back on critical failures

4.7 Audit Logging

4.7.1 Logged Events

The system records:

- Optimization start/completion
- Parameter changes (budget, time limits)
- Manual assignment adjustments
- Data imports and exports
- User actions

4.7.2 Audit Trail Benefits

- Transparency for stakeholders
- Debugging and troubleshooting
- Compliance and accountability
- Performance analysis

4.8 OptaPlanner and Constraint Solving

4.8.1 Why OptaPlanner?

OptaPlanner was selected for several reasons:

- **Mature constraint solver:** Production-ready with years of development and optimization
- **Java integration:** Seamlessly integrates with Spring Boot backend
- **Flexible scoring:** Supports complex hard/soft constraint modeling
- **Multiple algorithms:** Includes various local search metaheuristics (Tabu Search, Late Acceptance, Simulated Annealing)
- **Active community:** Well-documented with examples and support

4.8.2 Core OptaPlanner Concepts

Planning Entity:

- Objects being optimized (PlannedInternship for Phase 1, StudentInternshipDemand for Phase 2)
- Contain planning variables (fields OptaPlanner can change)

Planning Variable:

- Fields within planning entities that OptaPlanner modifies
- Examples: `assignedTeacher`, `active`, `assignedInternship`

Planning Solution:

- Container holding all planning entities and problem facts
- Includes score representing solution quality

Score:

- Numerical representation of solution quality
- Format: HardSoftScore (hard score, soft score)
- Example: 0hard/-250soft means all hard constraints satisfied, soft constraint violations total -250

4.8.3 Local Search Metaheuristics

OptaPlanner uses local search algorithms that:

- Start with an initial solution (often randomly generated)
- Iteratively make small changes (moves) to improve the score
- Use strategies to avoid getting stuck in local optima:
  - **Tabu Search:** Maintains a "tabu list" of recent moves to avoid cycling
  - **Late Acceptance:** Accepts moves that improve on solutions from several iterations ago
  - **Simulated Annealing:** Probabilistically accepts worse moves early, becoming more selective over time

Chapter 5

Development Process & Results

5.1 Development Methodology

The project followed an Agile/Scrum process:

- **Sprint-based development:** Five sprints, each lasting two weeks
- **Sprint planning:** Requirements clarification, task breakdown, and estimation
- **Daily coordination:** Regular team communication via Discord and GitLab
- **Sprint reviews:** Demonstrations to teaching assistant and client feedback
- **Retrospectives:** Continuous process improvement

5.2 Evolution of the System

5.2.1 Sprint Overview

Sprints 1-2: Foundation

- Initial data model design (students, teachers, schools)
- Basic CRUD operations and REST API endpoints
- Simple constraint modeling and first optimization attempt
- Frontend scaffolding and core UI components

Sprints 3-4: Constraint Refinement

- Distance optimization for PDP internships
- Zone and public transport constraints
- Teacher diversity and workload balancing
- Integration of client feedback on constraint priorities

Sprints 5-6: Multi-Phase Optimization

- Separation into Phase 0/1/2 architecture
- Preservation constraint for semester continuity
- Performance tuning and solver configuration
- Budget and capacity constraint refinements

Sprints 7+: Refinement and Testing

- Edge case handling and validation
- UI polish and usability improvements
- Documentation and deployment preparation

5.2.2 Key Milestones

- **Week 3:** First working end-to-end assignment (single-phase approach)
- **Week 5:** Distance optimization integrated after client feedback
- **Week 7:** Multi-phase architecture implemented, significant performance improvement
- **Week 9:** Preservation constraint for semester continuity added
- **Week 10:** System ready for client testing with real data

5.3 Team Collaboration

5.3.1 Internal Coordination

- Clear role distribution: backend specialists, frontend specialists, and integration coordinators
- Code review practices via GitLab merge requests
- Weekly team meetings for synchronization
- Pair programming for complex optimization logic

5.3.2 External Coordination

- Regular meetings with teaching assistant for guidance
- Client feedback sessions to validate requirements
- Coordination with other teams to share insights (without sharing code)

5.4 Quality Metrics

5.4.1 Code Quality

- GitLab activity: **XXX commits** over 10 weeks
- TeamScale code quality score: **X.X/5.0**
- Code review coverage: All merge requests reviewed by at least one team member

5.4.2 Testing

- Unit tests for core constraint logic
- Integration tests for API endpoints
- End-to-end testing with realistic datasets
- Manual testing and validation by client

5.5 Challenges and Solutions

5.5.1 Technical Challenges

Challenge 1: OptaPlanner Caching Issues

- *Problem:* Incremental score calculation caused incorrect constraint evaluation
- *Solution:* Switched to `EasyScoreCalculator` for transparent, cache-free scoring

Challenge 2: Performance with Large Problem Instances

- *Problem:* Single-phase approach took excessive time (10+ minutes for 400 students)
- *Solution:* Multi-phase architecture reduced complexity, achieving solutions in under 2 minutes

Challenge 3: Distance Calculation Accuracy

- *Problem:* Straight-line distance didn't reflect real travel feasibility
- *Solution:* Implemented Haversine formula and integrated public transport zones as proxy for accessibility

5.5.2 Requirement Changes

Change 1: Teacher Workload Flexibility

- *Initial requirement:* Teachers assigned exactly 2 internships
- *Revised requirement:* Teachers can have 0, 1, 2, or 4 internships based on historical data analysis

- *Impact:* Relaxed hard constraint, added soft constraint preference for 2 internships

Change 2: Semester Continuity Preservation

- *New requirement (Sprint 5):* When reoptimizing for a new semester, preserve teacher assignments when possible
- *Implementation:* Added baseline preservation mechanism and soft constraint rewarding continuity

5.6 Performance Analysis

5.6.1 Phase 1 Solver Performance

Problem size: 420 planned internship slots, 100 teachers, 100 schools
Metrics:

- Average runtime: **60-90 seconds** (configurable time limit)
- Hard constraint satisfaction: **100%** in all tested scenarios
- Soft score improvement: Consistent convergence within time budget

5.6.2 Phase 2 Solver Performance

Problem size: 420 student demands, 200+ activated internship slots
Metrics:

- Average runtime: **30-60 seconds** (configurable time limit)
- Student assignment success rate: **100%** in all tested scenarios
- Average PDP distance: **¡15 km** per student

5.7 Solution Quality

5.7.1 Constraint Satisfaction

Hard constraints: Zero violations across all test scenarios with realistic data.
Soft constraints:

- Teacher workload preference: **85%** of active teachers assigned exactly 2 internships
- Distance minimization: **90%** of PDP students assigned within 20 km of home
- Subject preference matching: **80%** of ZSP students matched to preferred courses

5.7.2 Stakeholder Feedback

Client satisfaction:

- Positive feedback on assignment quality and geographic distribution
- Appreciation for multi-phase approach reflecting real planning workflow
- Requests for minor UI improvements (implemented in final sprint)

Usability observations:

- Learning curve manageable for non-technical users
- Manual adjustment features valuable for edge cases
- Export functionality useful for communication with schools

5.8 Limitations

Despite the system's strengths, some limitations remain:

- **Feasibility not guaranteed:** In extreme edge cases with unusual constraint combinations, the system may produce near-feasible but not perfectly feasible solutions
- **Trade-offs in multi-objective optimization:** Balancing competing objectives (e.g., distance vs. teacher preference) requires manual score weight tuning
- **Computational constraints:** Very large problem instances (1000+ students) may require longer solver runtimes or more powerful hardware
- **Manual data entry:** Initial data input (students, teachers, schools) requires significant manual effort; integration with university systems would reduce this burden

Chapter 6

Conclusion

6.1 Summary

This project delivered a comprehensive internship assignment optimization system for the University of Passau's *Praktikumsamt*. The system addresses the complex challenge of coordinating hundreds of teaching internships across multiple schools, teachers, and students while respecting numerous constraints and optimizing for quality.

Key achievements:

- **Multi-phase optimization architecture:** Successfully separated teacher assignment from student assignment, reducing computational complexity and reflecting real-world planning workflows
- **Constraint satisfaction solving:** Implemented sophisticated constraint modeling using OptaPlanner, achieving 100% hard constraint satisfaction in tested scenarios
- **Distance optimization:** Minimized student travel distances for PDP internships through bidirectional matching and centroid-based positioning
- **Complete web application:** Delivered a full-stack system with backend API, optimization engine, and user-friendly frontend
- **Real-world validation:** Tested with realistic data and validated by the client for practical deployment

6.2 Reflection on Optimization Goals

6.2.1 Were Optimization Objectives Met?

The system successfully achieves its core objectives:

- **Feasibility:** Consistently generates assignment plans satisfying all hard constraints in realistic scenarios
- **Quality optimization:** Effectively balances competing soft constraints (distance, workload, preferences) to produce high-quality solutions
- **Efficiency:** Completes optimization in acceptable timeframes (under 2 minutes total for both phases)

6.2.2 Multi-Phase Approach Effectiveness

The decision to separate teacher assignment (Phase 1) from student assignment (Phase 2) proved highly effective:

- **Performance gain:** Reduced total runtime from 10+ minutes (single-phase) to under 2 minutes (multi-phase)
- **Conceptual clarity:** Each phase has a clear, focused objective, simplifying constraint modeling and debugging
- **Flexibility:** Enables independent re-optimization of Phase 2 for what-if scenarios without re-running Phase 1
- **Real-world alignment:** Mirrors the actual planning workflow used by the *Praktikumsamt*

6.2.3 Balance Between Automation and Manual Control

The system strikes a good balance:

- **Automation:** Core assignment optimization is fully automated, reducing manual effort significantly
- **Control:** Users retain ability to review, adjust, and override assignments when needed
- **Transparency:** Audit logs and score explanations help users understand why particular assignments were made

6.3 Lessons Learned

6.3.1 Technical Insights

- **Constraint solver complexity:** OptaPlanner is powerful but requires deep understanding of its internals (caching, scoring) to use effectively
- **Problem decomposition:** Breaking large optimization problems into phases is often more effective than solving them monolithically
- **Score weight tuning:** Finding the right balance between competing soft constraints requires iterative experimentation and client feedback
- **Real-world constraints matter:** Distance calculations, zone restrictions, and other practical considerations significantly impact solution quality

6.3.2 Process Learnings

- **Agile works for complex problems:** Iterative development allowed us to adapt to evolving requirements and client feedback
- **Client engagement is crucial:** Regular feedback sessions ensured we were solving the right problem in the right way

- **Code quality pays off:** Investing in code review and testing early saved debugging time later
- **Documentation matters:** Clear comments and documentation were essential for team coordination and knowledge transfer

6.3.3 Collaboration Experiences

- **Role specialization:** Having team members focus on backend, frontend, or optimization allowed for deeper expertise
- **Communication overhead:** Regular synchronization was necessary but sometimes time-consuming; tools like GitLab and Discord helped
- **Learning from peers:** Informal exchanges with other teams (without sharing code) provided valuable alternative perspectives

6.4 Future Work

6.4.1 Technical Enhancements

- **Better multi-objective optimization:** Implement Pareto frontier analysis to present multiple high-quality solutions with different trade-offs
- **Machine learning for constraint tuning:** Use historical assignment data to automatically learn optimal score weights
- **Real-time reoptimization:** Enable incremental updates when individual constraints change (e.g., teacher becomes unavailable) without full re-optimization
- **Advanced distance metrics:** Integrate real routing APIs (Google Maps, OpenStreetMap) for accurate travel time and public transport feasibility

6.4.2 Feature Extensions

- **Scenario comparison:** Allow users to generate and compare multiple assignment plans side-by-side
- **What-if analysis:** Enable users to explore "what if" questions (e.g., "What if we increase the budget by 10%?")
- **Advanced reporting:** Generate detailed reports for stakeholders (schools, teachers, students) with personalized assignment information
- **Visualization improvements:** Add geographic map views showing assignment distributions and distance patterns
- **Integration with university systems:** Connect to student information systems for automatic data import, reducing manual entry burden

6.4.3 Process Improvements

- **Automated testing:** Expand test coverage, especially for complex constraint interactions
- **Performance optimization:** Profile and optimize bottlenecks for even larger problem instances
- **Scalability:** Architect for horizontal scaling if the system is extended to multiple universities or regions
- **Deployment automation:** Implement CI/CD pipelines for streamlined deployment and updates

6.5 Final Remarks

This project successfully demonstrated that complex, real-world assignment problems can be effectively addressed through systematic constraint satisfaction and optimization techniques. The system delivers tangible value to the *Praktikumsamt*, reducing administrative burden while improving assignment quality.

Beyond the technical solution, the project provided invaluable experience in collaborative software engineering, agile development, client engagement, and problem-solving under real-world constraints. These skills and insights will serve us well in future software engineering endeavors.

We are proud of what we accomplished in ten weeks and grateful to our client, teaching assistant, and team members for making this project a success.

Bibliography

Appendix A

Additional Materials

A.1 Key Code Samples

A.2 Solver Configuration Examples